

Ứng dụng thuật toán chia để trị vào bài toán đường đi trên cây

Qì Zichao
Trường Trung học Yali, Trường Sa

Luận văn đội tuyển dự tuyển quốc gia IOI 2009

Nguồn gốc: Ứng dụng thuật toán chia để trị vào bài toán đường đi trên cây

IOI China candidate team papers / 2009 / Qì Zichao

Abstract

Cây là một cấu trúc dữ liệu đặc biệt và có vai trò rất quan trọng trong Tin học. Trong các kỳ thi, bài toán về cây xuất hiện rất thường xuyên. Bài viết chọn một số bài toán gần đây liên quan tới đường đi trên cây, rồi thông qua các ví dụ để trình bày cách dùng chia để trị cho lớp bài này.

Từ khóa. Cây; đường đi; phân rã đường; chia để trị; cấu trúc dữ liệu.

Contents

1	Mở đầu	1
2	Chia để trị trên cây	1
2.1	Chia để trị theo đỉnh	2
2.2	Chia để trị theo cạnh	2
2.3	Phân tích hiệu quả	2
2.4	Ví dụ 1: Đếm cặp đỉnh trên cây	3
2.5	Ví dụ 2: Free Tour 2	4
2.6	Tiểu kết phần chia để trị	5
3	Phân rã đường trên cây	5
3.1	Ví dụ 3: Query On a Tree	5
3.2	Đưa vào phân rã đường	5
3.3	Phân rã đường theo cạnh nặng và cạnh nhẹ	5
3.4	Ví dụ 4: Cây đen trắng	6
3.5	Tiểu kết về phân rã đường	6
3.6	Liên hệ giữa phân rã đường và chia để trị trên cây	6
3.7	Ví dụ 5: Query On a Tree IV	7
3.8	Tiểu kết phần phân rã đường	8

4	Thảo luận thêm về chia để trị trên cây	8
4.1	Cải thiện độ phức tạp của chia để trị theo cạnh	8
4.2	Dùng chia để trị theo cạnh cho ví dụ 2	9
4.3	Dùng chia để trị theo cạnh cho ví dụ 5	9
5	Tổng kết	9

1 Mở đầu

Cây được định nghĩa là một đồ thị liên thông không có chu trình. Nó có các tính chất cơ bản sau:

1. Nếu bỏ đi một cạnh bất kỳ của cây, đồ thị thu được sẽ không còn liên thông.
2. Nếu thêm một cạnh bất kỳ vào cây, đồ thị thu được chắc chắn có chu trình.
3. Giữa mọi cặp đỉnh U, V của cây tồn tại đúng một đường đi.

Vì cây có những đặc điểm mà đồ thị tổng quát không có, nó được dùng rất rộng rãi trong thi lập trình. Đặc biệt, các bài toán lấy đường đi trên cây làm đối tượng truy vấn xuất hiện nhiều trong các kỳ thi. Mỗi thí sinh hướng tới OI hoặc ACM đều nên nắm vững các thuật toán cho lớp bài này.

Chia để trị là tư tưởng chia một bài toán thành các bài toán con độc lập có kích thước nhỏ hơn để xử lý riêng. Ta thường gặp chia để trị trên cấu trúc tuyến tính; trong bài viết này, ta xét việc dùng chia để trị trên cấu trúc cây, gọi là **chia để trị trên cây**.

Chia để trị thường gắn với hiệu quả cao, và chia để trị trên cây chính là một nhóm thuật toán hiệu quả để giải bài toán đường đi trên cây.

2 Chia để trị trên cây

Phần này đưa ra hai dạng chia để trị thường gặp trên cây.

2.1 Chia để trị theo đỉnh

Trước hết chọn một đỉnh, xem nó là gốc để biến cây vô hướng thành cây có gốc. Sau đó đệ quy xử lý từng cây con có gốc là con của đỉnh gốc đó.

Nói cách khác, mỗi bước ta xóa một đỉnh được chọn; các thành phần liên thông còn lại trở thành những bài toán con độc lập.

2.2 Chia để trị theo cạnh

Trong cây, chọn một cạnh. Khi bỏ cạnh này đi, cây ban đầu tách thành hai cây không giao nhau; ta đệ quy xử lý hai cây đó.

2.3 Phân tích hiệu quả

Trước hết cần xét cách chọn đỉnh hoặc cạnh. Với chia để trị theo đỉnh, ta chọn một đỉnh sao cho sau khi xóa nó, kích thước của cây con lớn nhất là nhỏ nhất. Đỉnh này được gọi là **trọng tâm của cây**. Với chia để trị theo cạnh, ta muốn chọn cạnh sao cho hai cây tách ra có kích thước càng cân bằng càng tốt; trong

bài viết này gọi tạm cạnh đó là **cạnh trung tâm**.¹ Cả hai bài toán chọn này đều có thể giải bằng quy hoạch động trên cây trong thời gian $\mathcal{O}(N)$, với N là số đỉnh của cây.

Đối với chia để trị trên cây, độ sâu đệ quy thường quyết định hiệu quả của thuật toán. Ta phân tích độ sâu đệ quy xấu nhất của hai dạng trên.

Định lý 1. Tồn tại một đỉnh sao cho mọi cây con thu được sau khi xóa đỉnh đó đều có không quá $N/2$ đỉnh.

Chứng minh. Giả sử U là trọng tâm của cây, kề với $V_1, V_2, \dots, V_k$. Ký hiệu $\text{Size}(X)$ là số đỉnh của cây con có gốc X . Gọi V là đỉnh có giá trị Size lớn nhất trong $V_1, \dots, V_k$.

Ta chứng minh phản chứng. Giả sử $\text{Size}(V) > N/2$. Bây giờ xét trường hợp chọn V làm gốc, và ký hiệu $\text{Size}'(X)$ là kích thước cây con của X trong cách chọn gốc mới này.

Trong hình minh họa của bản gốc, phần nằm dưới các con khác của V nhỏ hơn phần cây con cũ của V , nên với các nhánh loại đó ta có $\text{Size}'(T_i) < \text{Size}(V)$. Phần còn lại đi qua U có kích thước

$$\text{Size}'(U) = N - \text{Size}(V) < N/2 < \text{Size}(V).$$

Như vậy nếu chọn V thay cho U , thành phần lớn nhất sẽ nhỏ hơn $\text{Size}(V)$, mâu thuẫn với giả thiết U là trọng tâm. Định lý được chứng minh.

Từ định lý 1 suy ra trong chia để trị theo đỉnh, mỗi bước kích thước bài toán con lớn nhất giảm ít nhất một nửa. Vì thế độ sâu đệ quy xấu nhất là $\mathcal{O}(\log N)$; cận này đạt được khi cây là một đường thẳng.

Định lý 2. Nếu mọi đỉnh của một cây đều có bậc không vượt quá D , thì tồn tại một cạnh sao cho hai cây tách ra có số đỉnh nằm trong đoạn

$$\left[\frac{N}{D+1}, \frac{ND}{D+1} \right],$$

với $N \geq 2$.

Chứng minh. Không mất tính tổng quát, coi D là bậc lớn nhất của mọi đỉnh.

Khi $D = 1$, mệnh đề hiển nhiên. Khi $D > 1$, giả sử phương án tối ưu là cạnh $U - V$, và hai cây con có gốc U, V lần lượt có kích thước S và $N - S$. Ta giả sử $S \geq N - S$.

Gọi X là con của U có cây con lớn nhất, và xét phương án chọn cạnh $X - U$. Sau khi bỏ cạnh $X - U$, gọi P là kích thước cây con có gốc X . Rõ ràng

$$P \geq \frac{S-1}{D-1}.$$

Do $P < S$ và cạnh $U - V$ là tối ưu, ta phải có $N - P \geq S$. Kết hợp với bất đẳng thức trên, suy ra

$$S \leq \frac{(D-1)N+1}{D}.$$

Vì $N \geq D+1$, ta có

$$S \leq \frac{ND}{D+1}.$$

Định lý được chứng minh.

Từ định lý 2, nếu D là hằng số thì chia để trị theo cạnh cũng có độ sâu đệ quy xấu nhất $\mathcal{O}(\log N)$. Tuy nhiên trong bài toán tổng quát, D có thể rất lớn, thậm chí đạt $\mathcal{O}(N)$. Khi đó thuật toán chia để trị theo cạnh có thể kém hiệu quả. Vì vậy trong phần này ta chủ yếu xét chia để trị theo đỉnh.

¹Tác giả không thấy tên gọi chuẩn của cạnh này trong tài liệu tham khảo, nên tạm gọi như vậy.

2.4 Ví dụ 1: Đếm cặp đỉnh trên cây

Tóm tắt đề bài. Cho một cây có trọng số gồm N đỉnh, với $1 \leq N \leq 10000$. Định nghĩa $\text{dist}(u, v)$ là độ dài đường đi ngắn nhất giữa u và v , tức tổng trọng số các cạnh trên đường đi. Cho thêm K , với $1 \leq K \leq 10^9$. Với hai đỉnh khác nhau a, b , nếu $\text{dist}(a, b) \leq K$, ta gọi (a, b) là một cặp đỉnh hợp lệ. Hãy đếm số cặp đỉnh hợp lệ.²

Phân tích. Nếu dùng DFS thông thường để xét mọi cặp đỉnh, độ phức tạp lên tới $\mathcal{O}(N^2)$. Một quy hoạch động $\mathcal{O}(NK)$ cũng không thể chạy trong giới hạn thời gian vì K quá lớn.

Ta dùng nhận xét: một đường đi hoặc đi qua gốc hiện tại, hoặc nằm hoàn toàn trong một cây con. Điều này gợi ý dùng chia để trị.

Trường hợp đường đi nằm trong cây con được xử lý bằng đệ quy. Ta chỉ cần xét cách đếm các đường đi đi qua gốc. Ký hiệu $\text{Depth}(i)$ là độ dài đường đi từ đỉnh i tới gốc. Ký hiệu $\text{Belong}(i) = X$, trong đó X là một con của gốc và i nằm trong cây con có gốc X . Khi đó số cần đếm là số cặp (i, j) thỏa

$$\text{Depth}(i) + \text{Depth}(j) \leq K, \quad \text{Belong}(i) \neq \text{Belong}(j).$$

Ta viết lại thành

$$\begin{aligned} & \#\{(i, j) \mid \text{Depth}(i) + \text{Depth}(j) \leq K, \text{Belong}(i) \neq \text{Belong}(j)\} \\ &= \#\{(i, j) \mid \text{Depth}(i) + \text{Depth}(j) \leq K\} \\ &\quad - \#\{(i, j) \mid \text{Depth}(i) + \text{Depth}(j) \leq K, \text{Belong}(i) = \text{Belong}(j)\}. \end{aligned}$$

Cả hai phần đều quy về bài toán: cho một dãy A , đếm số cặp (i, j) thỏa $A_i + A_j \leq K$. Sau khi sắp xếp A , dùng hai con trỏ và tính đơn điệu để xử lý trong $\mathcal{O}(N)$ cho mỗi mức chia. Do đó bài toán được giải trong thời gian $\mathcal{O}(N \log^2 N)$ nếu sắp xếp lại ở mỗi mức, và có thể cải thiện trong cài đặt bằng cách quản lý danh sách sâu cẩn thận. Kết luận chính ở đây là chia để trị trên cây giảm đáng kể so với xét mọi cặp.

2.5 Ví dụ 2: Free Tour 2

Tóm tắt đề bài. Cho một cây có trọng số gồm N đỉnh. Mỗi đỉnh thuộc một trong hai loại: đen hoặc trắng. Hãy tìm một đường đi sao cho số đỉnh đen trên đường đi không vượt quá K , đồng thời độ dài đường đi là lớn nhất. Ràng buộc:

$$N \leq 200000.$$

Bài gốc là SPOJ 1825, *Free Tour II*.

Phân tích. Vì N có thể tới 200000, thuật toán ngây thơ khó vượt qua giới hạn thời gian. Đối tượng cần duy trì vẫn là đường đi trên cây, nên ta thử dùng chia để trị trên cây.

Tương tự ví dụ trước, ở một mức chia ta chỉ cần xử lý các đường đi qua gốc; các đường đi còn lại nằm trong cây con và được xử lý đệ quy.

Ký hiệu $G(i, j)$ là độ dài tốt nhất của đường đi bắt đầu từ người con thứ i của gốc đi xuống một đỉnh trong cây con đó, với điều kiện đường đi này có không quá j đỉnh đen. Ký hiệu $\text{Dep}(i)$ là số đỉnh đen lớn nhất trên một đường đi từ người con thứ i của gốc tới một đỉnh trong cây con của nó.

Khi $j > \text{Dep}(i)$, ta có

$$G(i, j) = G(i, \text{Dep}(i)).$$

²POJ 1741, *Tree*. Bản gốc cũng ghi nguồn LTC_Man_Contest E.

Vì thế chỉ cần giữ các giá trị $j \leq \text{Dep}(i)$. Các giá trị G có thể được tính bằng DFS trong tổng thời gian tuyến tính ở mức chia hiện tại.

Mục tiêu là tính

$$\text{Max}\{G(u, L_1) + G(v, L_2)\},$$

trong đó $u \neq v$ và

$$L_1 + L_2 = K - \text{Black}[\text{Root}].$$

Nếu gốc là đỉnh đen thì $\text{Black}[\text{Root}] = 1$, ngược lại bằng 0.

Bài toán kết hợp này có thể làm bằng cây cân bằng trong $\mathcal{O}(N \log N)$ cho một mức. Tuy nhiên ở đây có một cách không cần cấu trúc dữ liệu cao cấp. Điều kiện $u \neq v$ có thể thay bằng $u > v$ nếu ta quét các con của gốc theo một thứ tự. Khi đang xét u , điều cốt lõi là duy trì

$$\text{Max}\{G(v, L_2) \mid v < u\}.$$

Giả sử đã có giá trị trên cho $v < u - 1$; ta hợp nhất nó với dãy $G(u - 1, \cdot)$ để thu được giá trị cho $v < u$:

$$\text{Max}\{G(v, L_2) \mid v < u\} = \max(\text{Max}\{G(v, L_2) \mid v < u - 1\}, G(u - 1, L_2)).$$

Chú ý rằng ta không tính rõ toàn bộ dãy vô hạn $G(u, \cdot)$, mà chỉ giữ một đoạn đầu hữu ích. Nếu hai dãy có độ dài lưu trữ lần lượt là $\text{Len}_1, \text{Len}_2$, phép hợp nhất tốn $\mathcal{O}(\max(\text{Len}_1, \text{Len}_2))$.

Gọi số con của gốc là TotChild . Nếu xử lý trực tiếp theo thứ tự bất kỳ, số lần tính toán có thể là

$$\text{Dep}(1) + \max\{\text{Dep}(1), \text{Dep}(2)\} + \dots + \max\{\text{Dep}(1), \dots, \text{Dep}(\text{TotChild})\},$$

xấu nhất đạt $\mathcal{O}(N^2)$. May mắn là thứ tự các con của gốc không ảnh hưởng tới đáp án. Vì vậy ta sắp xếp các con theo khóa $\text{Dep}(i)$, rồi xử lý theo thứ tự tăng dần. Khi đó tổng số thao tác trở thành

$$\text{Dep}(1) + \text{Dep}(2) + \dots + \text{Dep}(\text{TotChild}) \leq N.$$

Vì số cạnh là $\mathcal{O}(N)$, chi phí sắp xếp qua toàn bộ thuật toán không vượt quá $\mathcal{O}(N \log N)$. Do đó độ phức tạp tổng thể là $\mathcal{O}(N \log N)$.

2.6 Tiêu kết phần chia để trị

Phần này trình bày cách dùng chia để trị trên cây cho một lớp bài toán tính toán và thống kê đường đi. Từ hai ví dụ trên có thể thấy hiệu quả của thuật toán này.

Trọng tâm của phần này là chia để trị theo đỉnh. Người đọc có thể nhận thấy trong chia để trị theo đỉnh, số cây con sinh ra ở một bước có thể khá lớn, làm việc duy trì thông tin trở nên khó hơn.

Nếu dùng chia để trị theo cạnh cho hai ví dụ trên, phân tích thuật toán sẽ đơn giản hơn vì mỗi bước chỉ cần duy trì hai cây con. Đây là ưu điểm lớn của chia để trị theo cạnh về mặt tư duy thiết kế. Tuy nhiên như đã phân tích, độ phức tạp xấu nhất quá lớn là nhược điểm chí mạng của cách này. Điều đó không có nghĩa chia để trị theo cạnh vô dụng; ở phần sau ta sẽ thảo luận cách bảo đảm độ phức tạp của nó.

3 Phân rã đường trên cây

3.1 Ví dụ 3: Query On a Tree

Tóm tắt đề bài. Cho một cây gồm N đỉnh, mỗi cạnh có một trọng số. Cần mô phỏng hai loại thao tác:

1. đổi trọng số của một cạnh;

2. hỏi cạnh có trọng số lớn nhất trên đường đi giữa hai đỉnh U, V .

Ràng buộc:

$$N \leq 10000.$$

Bài gốc là SPOJ 375, *QTREE*.

Phân tích. Nếu mô phỏng trực tiếp, truy vấn loại hai có độ phức tạp kỳ vọng $\mathcal{O}(\log N)$ trong một số mô hình ngẫu nhiên, nhưng trường hợp xấu nhất có thể đạt $\mathcal{O}(N)$. Khi số truy vấn lớn, cách này không thể đáp ứng giới hạn.

3.2 Đưa vào phân rã đường

Hình dạng của cây không đổi, chỉ có trọng số cạnh thay đổi. Vì vậy ta nghĩ tới việc phân rã các đường đi của cây. Dưới đây là một phương pháp rất thường dùng trong thực tế: phân rã nặng nhẹ.

3.3 Phân rã đường theo cạnh nặng và cạnh nhẹ

Ta chia cạnh trong cây thành hai loại: cạnh nặng và cạnh nhẹ. Ký hiệu $\text{Size}(U)$ là số đỉnh trong cây con có gốc U . Gọi V là con của U có Size lớn nhất. Khi đó cạnh (U, V) được gọi là **cạnh nặng**; các cạnh còn lại từ U xuống con của nó là **cạnh nhẹ**.

Phân rã nặng nhẹ có các tính chất sau.

Tính chất 1. Nếu (U, V) là cạnh nhẹ thì

$$\text{Size}(V) \leq \frac{\text{Size}(U)}{2}.$$

Tính chất 2. Trên đường đi từ gốc tới một đỉnh bất kỳ, số cạnh nhẹ không vượt quá $\mathcal{O}(\log N)$.

Tính chất 3. Gọi một đường là **đường nặng** nếu nó chỉ gồm các cạnh nặng. Khi đó trên đường đi từ mỗi đỉnh tới gốc có không quá $\mathcal{O}(\log N)$ cạnh nhẹ và $\mathcal{O}(\log N)$ đường nặng.

Quay lại bài toán. Ta phân rã cây bằng cạnh nặng nhẹ. Với một truy vấn đường đi, ta xử lý riêng hai đoạn từ hai đầu mút tới tổ tiên chung gần nhất. Theo tính chất 3, mỗi đoạn được tách thành nhiều nhất $\mathcal{O}(\log N)$ cạnh nhẹ và $\mathcal{O}(\log N)$ đường nặng. Vì thế chỉ cần biết cách duy trì hai loại đối tượng này.

Với cạnh nhẹ, có thể xử lý trực tiếp. Với đường nặng, dùng cây đoạn để duy trì giá trị lớn nhất. Khi đó độ phức tạp của thao tác cập nhật là $\mathcal{O}(\log N)$, còn truy vấn là $\mathcal{O}(\log^2 N)$.³ Như vậy thuật toán có thể vượt qua toàn bộ dữ liệu của bài.

3.4 Ví dụ 4: Cây đen trắng

Tóm tắt đề bài. Ban đầu ta có một cây gồm N đỉnh, tất cả đều màu trắng. Cần xử lý C lệnh:

1. lệnh sửa đổi: đổi màu một đỉnh cho trước, trắng thành đen hoặc đen thành trắng;
2. lệnh truy vấn: hỏi trên đường đi từ đỉnh 1 tới một đỉnh cho trước, đỉnh đen đầu tiên có chỉ số nào.

³Có thuật toán có độ phức tạp thấp hơn và chạy nhanh hơn trong thực tế; xem tài liệu tham khảo về các cách giải QTREE.

Ràng buộc:

$$N \leq 1000000, \quad C \leq 1000000.$$

Bài gốc là bài 3 vòng chung kết Astar 2008.

Phân tích. Thoạt nhìn, bài này không dễ xử lý. Cụm “đỉnh đen đầu tiên trên đường đi tới một đỉnh cho trước” dường như không có tính chất đặc biệt rõ ràng, và cũng khó tìm một cách tổ chức dữ liệu trực tiếp để duy trì nó.

Vì hình dạng của cây không đổi, còn đối tượng cần duy trì là đường đi từ một đỉnh tới gốc, ta xét dùng phân rã đường. Khác với ví dụ trước, ở đó ta duy trì giá trị lớn nhất của các cạnh; ở đây đối tượng được duy trì là các đỉnh.

Ta vẫn có thể dùng phân rã đường. Mỗi đỉnh thuộc đúng một đường nặng, nên ở bài này chỉ cần xét các đường nặng mà không cần xử lý riêng cạnh nhẹ. Như vậy số loại đối tượng phải duy trì còn ít hơn ví dụ trước. Duy trì một đường nặng tương đương với xử lý một bài toán trên cấu trúc tuyến tính; dùng heap hoặc cây đoạn đều đạt mục đích.

3.5 Tiểu kết về phân rã đường

Vai trò của phân rã đường là biến đổi đối tượng cần duy trì từ cấu trúc cây phức tạp thành nhiều cấu trúc tuyến tính quen thuộc. Nhờ vậy bài toán thường trở nên dễ xử lý hơn.

3.6 Liên hệ giữa phân rã đường và chia để trị trên cây

Vì phân rã đường và chia để trị trên cây đều giải được các bài toán đường đi trên cây, ta tự nhiên đặt câu hỏi: giữa chúng có liên hệ gì không? Câu trả lời là có.

Bản gốc minh họa một cây cùng phân rã nặng nhẹ của nó. Nếu giữ cách vẽ thông thường, khó thấy điểm đặc biệt. Nhưng nếu sắp lại các đỉnh theo số cạnh nhẹ trên đường từ đỉnh tới gốc, các đường nặng hiện ra như những khối được bóc tách theo tầng.

Khi đó có thể thấy phân rã đường và chia để trị trên cây rất giống nhau. Ở phần trước, ta có hai cách chia: xóa một đỉnh hoặc xóa một cạnh. Phân rã đường có thể hiểu là mỗi lần xóa một chuỗi đỉnh, tức một đường. Vì vậy phân rã đường có thể xem là chia để trị theo chuỗi. Cách chia này còn có một đặc điểm: con của đỉnh vừa bị xóa chắc chắn sẽ trở thành đỉnh đầu của chuỗi bị xóa ở lần sau. Đặc điểm đó giúp việc duy trì thông tin thuận tiện hơn. Do số cạnh nhẹ trên mọi đường từ gốc tới một đỉnh chỉ là $\mathcal{O}(\log N)$, độ phức tạp của thuật toán cũng được bảo đảm.

3.7 Ví dụ 5: Query On a Tree IV

Tóm tắt đề bài. Cho một cây gồm N đỉnh. Mỗi đỉnh có màu đen hoặc trắng. Cần mô phỏng hai loại thao tác:

1. đổi màu một đỉnh;
2. hỏi khoảng cách giữa hai đỉnh đen xa nhất.

Ràng buộc:

$$N \leq 100000,$$

và trị tuyệt đối của trọng số cạnh không vượt quá 1000. Bài gốc là SPOJ 2666, *Query On a Tree IV*.

Phân tích. Bài này có cùng nội dung với ZJOI 2007 *Hide*, nhưng khác ở chỗ trọng số cạnh có thể âm. Lời giải chuẩn của bài gốc dùng tính chất của dãy ngoặc, nhưng dãy ngoặc không dùng được khi cây có cạnh âm. Vì vậy ta cần cách khác.

Khoảng cách giữa hai đỉnh chính là độ dài đường đi. Ta thử giải bài này bằng phân rã đường. Thông thường, phân rã đường duy trì các đường từ đỉnh tới gốc, nên bài này thoát nhìn đường như không liên quan. Nhưng nếu nhớ rằng bản chất của phân rã đường là chia để trị theo chuỗi, bài toán trở nên tự nhiên hơn.

Xét một chuỗi trong phân rã. Ta muốn tính giá trị tốt nhất của các đường đi mà đỉnh cao nhất của đường nằm trong chuỗi này. Với một chuỗi có N đỉnh, ký hiệu $D(i)$ và $D_2(i)$ lần lượt là giá trị lớn nhất và lớn thứ hai của độ dài đường đi từ đỉnh thứ i của chuỗi đi xuống tới một đỉnh đen nào đó. Hai đường tương ứng chỉ được giao nhau tại đỉnh đầu. Nếu không tồn tại đường tới đỉnh đen, giá trị được coi là $-\infty$.

Mục tiêu là tìm độ dài lớn nhất của những đường đi có phần giao với chuỗi nằm trong đoạn $[1, N]$. Ta duy trì bằng cây đoạn. Với một đoạn $[L, R]$, lưu:

$$MaxL = \max\{Dist(L, i) + D(i)\},$$

$$MaxR = \max\{D(i) + Dist(i, R)\},$$

$$Opt = \text{độ dài lớn nhất của đường có phần giao với chuỗi trong } [L, R].$$

Ở đây $Dist(i, j)$ là khoảng cách giữa đỉnh thứ i và đỉnh thứ j trên chuỗi.

Giả sử nút P của cây đoạn ứng với $[L, R]$, có con trái Lc ứng với $[L, Mid]$ và con phải Rc ứng với $[Mid + 1, R]$. Khi đó:

$$MaxL(P) = \max\{MaxL(Lc), Dist(L, Mid + 1) + MaxL(Rc)\},$$

$$MaxR(P) = \max\{MaxR(Rc), MaxR(Lc) + Dist(Mid, R)\},$$

$$Opt(P) = \max\{Opt(Lc), Opt(Rc), MaxR(Lc) + MaxL(Rc) + Dist(Mid, Mid + 1)\}.$$

Vì

$$Dist(i, j) = Dist(1, j) - Dist(1, i),$$

nên $Dist(i, j)$ có thể tính trong $\mathcal{O}(1)$ nếu đã có mảng tiền tố trên chuỗi.

Với đoạn biên $[L, L]$, gọi nút cây đoạn là P , và gọi đỉnh thứ L của chuỗi là x . Khi đó:

$$MaxL(P) = D(L), \quad MaxR(P) = D(L).$$

Nếu x là đỉnh đen:

$$Opt(P) = \max\{D(L), D(L) + D_2(L)\}.$$

Nếu x là đỉnh trắng:

$$Opt(P) = D(L) + D_2(L).$$

Phần còn lại là duy trì D và D_2 . Gọi $C_1, \dots, C_k$ là các con của x không nằm cùng chuỗi, L_i là gốc cây đoạn của chuỗi chứa C_i , và $Cost(p)$ là trọng số cạnh (x, p) . Tập độ dài các đường đi từ x đi xuống tới một đỉnh đen là:

$$\{MaxL(L_i) + Cost(C_i), 0\} \quad \text{nếu } x \text{ là đỉnh đen,}$$

và

$$\{MaxL(L_i) + Cost(C_i)\} \quad \text{nếu } x \text{ là đỉnh trắng.}$$

Dùng heap để duy trì tập này, khi đó lấy $D(x)$ và $D_2(x)$ tốn $\mathcal{O}(1)$.

Với truy vấn hỏi đáp án, ta có thể dùng một heap lưu giá trị tối ưu của từng chuỗi, nên mỗi lần hỏi tốn $\mathcal{O}(1)$. Với thao tác đổi màu, một đỉnh chỉ ảnh hưởng tới $\mathcal{O}(\log N)$ chuỗi; mỗi chuỗi lại cần cập nhật heap và cây đoạn, vì vậy độ phức tạp mỗi lần đổi màu là $\mathcal{O}(\log^2 N)$.

Đáng chú ý là nếu dùng phương pháp thay đổi cấu trúc cây trình bày ở phần sau, độ phức tạp lập trình của bài này sẽ thấp hơn.

3.8 Tiểu kết phần phân rã đường

Qua phân tích sâu hơn về phân rã đường, ta thấy thuật toán này có thể hiểu là chia để trị theo chuỗi. Nhờ góc nhìn đó, một bài toán tưởng như không liên quan tới phân rã đường vẫn có thể được giải quyết trơn tru.

4 Thảo luận thêm về chia để trị trên cây

4.1 Cải thiện độ phức tạp của chia để trị theo cạnh

Trước hết, ta thử thay đổi tiêu chuẩn chọn cạnh. Tiếc là điều này không thể thay đổi độ phức tạp xấu nhất. Khi hình dạng cây giống một ngôi sao, dù chọn cạnh nào thì kết quả tách cũng rất lệch.

Phân tích ở trên cho thấy yếu tố quyết định là bậc của mỗi đỉnh. Ta đặt câu hỏi: liệu có thể biến đổi tương đương cây ban đầu thành một cây mới mà mọi đỉnh đều có bậc hằng số hay không?

Hãy nhìn lại ví dụ 5. Đối tượng được quan tâm là khoảng cách giữa hai đỉnh đen. Điều này gợi ý rằng ta có thể thêm các đỉnh trắng trung gian, miễn là khoảng cách giữa các đỉnh đen trong cây không đổi.

Bản gốc minh họa cách thay các cạnh từ một đỉnh tới nhiều con bằng một cấu trúc giống cây đoạn: các con được gom qua các đỉnh trắng trung gian. Nhờ thêm đỉnh trắng trên đường từ mỗi đỉnh tới các con của nó, ta thu được một cấu trúc tương đương. Cây đoạn trên N lá có không quá $2N$ đỉnh, nên một cây ban đầu có N đỉnh sau biến đổi có không quá bậc hằng số lần N đỉnh.

Quan trọng hơn, cây mới có một tính chất mà cây gốc không có: bậc của mỗi đỉnh không vượt quá 3. Do đó chia để trị theo cạnh trên cây mới có độ sâu $\mathcal{O}(\log N)$.

May mắn là cách thay đổi cấu trúc cây này có thể tổng quát hóa. Đỉnh trắng ở đây đóng vai trò đỉnh trung gian không ảnh hưởng tới kết quả. Với các bài toán khác, miễn là xác định được loại đỉnh trung gian trung tính tương ứng, ta có thể dùng cùng tư tưởng để đưa bậc cây về hằng số.

4.2 Dùng chia để trị theo cạnh cho ví dụ 2

Đây là ví dụ đã xuất hiện ở phần đầu. Khi đó ta dùng chia để trị theo đỉnh và đạt độ phức tạp $\mathcal{O}(N \log N)$.

Dùng chia để trị theo cạnh về cơ bản giống chia để trị theo đỉnh, nhưng chỉ cần xét hai cây con thay vì nhiều cây con. Vì thế thiết kế thuật toán đơn giản hơn. Sau khi biến đổi cây để bậc bị chặn, lời giải bằng chia để trị theo cạnh cũng đạt $\mathcal{O}(N \log N)$.

4.3 Dùng chia để trị theo cạnh cho ví dụ 5

Chia để trị theo cạnh chia đường đi thành hai loại: đường đi nằm trong một cây con, và đường đi đi qua cạnh trung tâm. Loại thứ nhất được xử lý bằng đệ quy; ta chỉ cần xét cách duy trì loại thứ hai.

Ký hiệu $\text{Dep}(x)$ là khoảng cách từ x tới gốc của cây con đang xét. Mục tiêu trở thành tính

$$\max\{\text{Dep}(x) + \text{Dep}(y) + \text{MidCost}\},$$

trong đó x, y đều là đỉnh đen và thuộc hai cây con khác nhau; MidCost là trọng số của cạnh trung tâm.

Ta chỉ cần dùng hai heap để duy trì các giá trị $\text{Dep}(x)$ của hai phía. Nhờ đó đáp án qua cạnh trung tâm được lấy trong $\mathcal{O}(1)$.

Với thao tác đổi màu, sau khi biến đổi cây để bậc bị chặn, mỗi đỉnh chỉ thuộc $\mathcal{O}(\log N)$ cây trong cây đệ quy. Do đó độ phức tạp mỗi lần đổi màu là $\mathcal{O}(\log^2 N)$.

Cách này đạt cùng bậc độ phức tạp với lời giải bằng phân rã đường, nhưng ý tưởng và cài đặt thường đơn giản hơn.

5 Tổng kết

Ta tóm tắt ngắn gọn các thuật toán đã trình bày.

Về hằng số thời gian, phân rã đường trên cây thường có hằng số nhỏ nhất; chia để trị theo đỉnh đứng sau; chia để trị theo cạnh thường có hằng số lớn hơn.

Về phạm vi ứng dụng, chia để trị theo chuỗi khác với chia để trị theo đỉnh hoặc theo cạnh. Đặc điểm của nó là con của đỉnh vừa bị xóa chắc chắn sẽ trở thành đầu của chuỗi bị xóa ở lần sau; chia để trị theo đỉnh hoặc theo cạnh không có tính chất này. Vì vậy phân rã đường thích hợp để duy trì các đỉnh hoặc cạnh trên một đường đi. Nếu đối tượng cần duy trì là độ dài đường đi, các cách chia theo đỉnh hoặc theo cạnh lại mạnh hơn.

So với chia để trị theo đỉnh, chia để trị theo cạnh thường làm việc thiết kế một thuật toán hiệu quả dễ nghĩ hơn rõ rệt, miễn là ta xử lý được vấn đề bậc lớn của cây.

Các thuật toán này có ưu điểm riêng. Khi giải bài, cần dựa vào cấu trúc cụ thể của dữ liệu và truy vấn để chọn cách thích hợp nhất.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi Tin học*.
2. Yang Zhe, bài tập đội tuyển quốc gia năm 2007, *Một số nghiên cứu về lời giải QTREE*.
3. Lou Tiancheng, bài tập đội tuyển quốc gia năm 2004, *Báo cáo lời giải đếm khoảng cách cặp đỉnh trên cây*.

Lời cảm ơn

Tác giả cảm ơn thầy Liao Xiaogang đã hướng dẫn trong quá trình viết bài, và cảm ơn bạn Liu Ying cùng bạn Sun Zheng từ Trường Trung học số 2 Hàng Châu đã giúp đỡ cho bài viết.

Phụ lục

Các liên kết bài gốc:

- Ví dụ 1: <http://acm.pku.edu.cn/JudgeOnline/problem?id=1741>
- Ví dụ 2: <http://www.spoj.pl/problems/FTOUR2/>
- Ví dụ 3: <http://www.spoj.pl/problems/QTREE/>
- Ví dụ 5: <http://www.spoj.pl/problems/QTREE4/>